

Symbolic Security

Message universe as a term algebra.

Let Σ be a cryptographic signature, e.g.

$$\Sigma \supseteq \{\text{Enc}, \text{Dec}, \text{pair}, \pi_1, \pi_2\}.$$

Messages are symbolic terms

$$t \in \mathcal{T}_\Sigma(X \cup \mathcal{N} \cup \mathcal{K}),$$

where X is a set of variables, $\mathcal{N}$ a set of nonces, and $\mathcal{K}$ a set of keys.

Cryptographic operators as ideal constructors/destructors.

Semantics is specified by an equational theory E (or rewrite system), e.g.

$$E = \{ \text{Dec}(\text{Enc}(m, k), k) = m \}.$$

Thus correctness is algebraic and exact, rather than probabilistic. Bit-level representation, entropy, cost, and implementation effects are abstracted away.

Adversary as symbolic deduction.

Given initial knowledge $\mathcal{K}_0$, the adversary derives new terms by closure under public operations and the equations in E :

$$\mathcal{K}_{i+1} = \text{cl}_E(\mathcal{K}_i), \quad \mathcal{K}^* = \bigcup_{i \geq 0} \mathcal{K}_i.$$

A ciphertext $\text{Enc}(m, k)$ yields m only when the adversary can derive the *entire* key term k .

Typical proof obligations.

Secrecy: $s \notin \mathcal{K}^*$ Authentication: $\neg \text{BadTrace}$

Privacy / indistinguishability: $P \approx Q$

Hence symbolic methods are especially effective for protocol logic, message-flow analysis, trace properties, equivalence properties, and state-machine inconsistencies.

Mathematical character

Algebraic, term-based, proof-search friendly, highly automatable.

Computational Security

Message universe as bitstrings.

Messages live in

$$m, c, k, r \in \{0, 1\}^*,$$

and primitives are randomized polynomial-time algorithms operating on bitstrings with an explicit security parameter λ .

Cryptographic primitive as an algorithmic tuple.

A symmetric encryption scheme is

$$\Pi = (\text{Gen}, \text{Enc}, \text{Dec}),$$

with

$$k \leftarrow \text{Gen}(1^\lambda), \quad c \leftarrow \text{Enc}_k(m; r), \quad m' \leftarrow \text{Dec}_k(c).$$

Correctness is quantified over all legal keys and messages:

$$\Pr[\text{Dec}_k(\text{Enc}_k(m; r)) = m] = 1.$$

Adversary as a PPT machine.

The attacker is a probabilistic polynomial-time algorithm

$$\mathcal{A} \in \text{PPT}.$$

Security is defined via experiments, reductions, and resource bounds. A canonical example is IND-CPA security:

$$\text{Adv}_\Pi^{\text{ind-cpa}}(\mathcal{A}, \lambda) = \left| \Pr[\text{Exp}_{\Pi, \mathcal{A}}^{\text{ind-cpa}}(\lambda) = 1] - \frac{1}{2} \right|.$$

Typical proof style.

One proves, for all $\mathcal{A} \in \text{PPT}$,

$$\text{Adv}_\Pi^{\text{ind-cpa}}(\mathcal{A}, \lambda) \leq \varepsilon(\lambda),$$

or in concrete security,

$$\forall \mathcal{A} \text{ with time } \leq t: \text{Adv}_\Pi(\mathcal{A}) \leq \varepsilon.$$

Proofs proceed by game hopping, reductions to hardness assumptions, and explicit accounting of probability and running time.

Mathematical character

Probabilistic, complexity-theoretic, reduction-based, closer to mainstream provable cryptography.

abstraction $\longleftrightarrow$ fidelity

Conceptual trade-off

Symbolic security replaces cryptographic computation by algebraic structure $(\mathcal{T}_\Sigma, E)$, gaining tractability and automation. *Computational security* models actual randomized algorithms on bitstrings, gaining fidelity to standard provable-security notions, but usually at a higher proof and automation cost.